

Informe Técnico: Analizador Léxico (C-Lexer)

1. ¿Qué tipo de autómata diseñaron, AFD o AFND? Se diseñó un Autómata Finito Determinista (AFD). Para el desarrollo de este analizador léxico, es necesario un modelo determinista donde cada carácter leído desde cualquier estado produce una única transición de estado posible, lo que garantiza una lectura lineal, eficiente y sin ambigüedades del código fuente.

2. ¿Cómo se diseñó el autómata escogido? El diseño del AFD se estructuró a partir de las reglas y componentes léxicos del lenguaje:

- Estado Inicial y Bifurcación: El autómata arranca en un estado base (q0) y transita a diferentes ramas funcionales dependiendo del primer carácter leído (bifurcando hacia estados de identificadores, números, operadores o literales).
- Manejo de Casos Especiales: Se diseñaron trayectorias de estados específicos para procesar directivas de preprocesador y evaluar caracteres de escape dentro de las cadenas de texto (error3_char_y_preproc.c y error4_escape_invalido.c).
- Estados de Error y Aceptación: Se definieron estados de aceptación final (para reconocer un *token* válido) y estados de error explícitos (estados "trampa") para capturar de inmediato anomalías sintácticas, tales como el uso de un carácter no reconocido (error1_caracter_invalido.c) o cadenas de texto que carecen de comillas de cierre (error2_string_sin_cerrar.c).

3. ¿Cómo se diseña la función de escaneo? La función de escaneo, que representa el motor principal del analizador (C-Lexer/src/scanner.c), se diseñó para simular en software el comportamiento del AFD:

- Ciclo de Consumo: Emplea un bucle iterativo que lee el archivo de entrada carácter por carácter. A través de estructuras de control de flujo (típicamente bloques switch-case o condicionales), evalúa el carácter actual contra el estado activo para realizar las transiciones matemáticas del autómata.
- Generación de Tokens: Una vez que la iteración alcanza un estado de aceptación (por ejemplo, al encontrar un espacio o delimitador que finaliza la secuencia actual), la función aísla el lexema (el texto reconocido), lo clasifica utilizando las estructuras definidas en las cabeceras (C-Lexer/include/token.h) y retorna el token al hilo principal (C-Lexer/src/main.c).